TEAM MEMBERS

Chaitanya - M01091125

Gurubasavaraj - M01093116

Muddassir - M01090890

AI Study Assistant Pro (RAG Chatbot) – Report

1. Introduction

In recent years, Artificial Intelligence (AI) and Large Language Models (LLMs) have significantly transformed how users interact with information. Traditional search methods often require users to manually read through long documents, which can be time-consuming and inefficient. To address this problem, intelligent systems such as AI-powered assistants can help users retrieve relevant information quickly and accurately.

This project, titled **"AI Study Assistant Pro"**, is designed to provide an interactive platform where users can upload study materials (in PDF format) and ask questions related to the content. The system uses **Retrieval-Augmented Generation (RAG)**, which combines information retrieval techniques with language models to generate context-aware answers.

The main objective of this project is to build a **functional AI application** that demonstrates the practical use of LLMs, vector databases, and conversational memory. Additionally, the system includes **basic guardrails** to ensure safe and relevant interactions.

This project was developed using Python and integrates multiple technologies such as Streamlit for the user interface, LangChain for orchestration, FAISS for vector storage, and HuggingFace models for natural language processing.

2. Objectives of the Project

The primary objectives of this project are:

- To develop a working AI-based chatbot application
- To implement Retrieval-Augmented Generation (RAG)
- To enable users to upload and query PDF documents
- To integrate conversational memory for better user interaction
- To apply basic guardrails to prevent misuse
- To create a clean and user-friendly interface using Streamlit

3. Technologies Used

The following technologies and tools were used in this project:

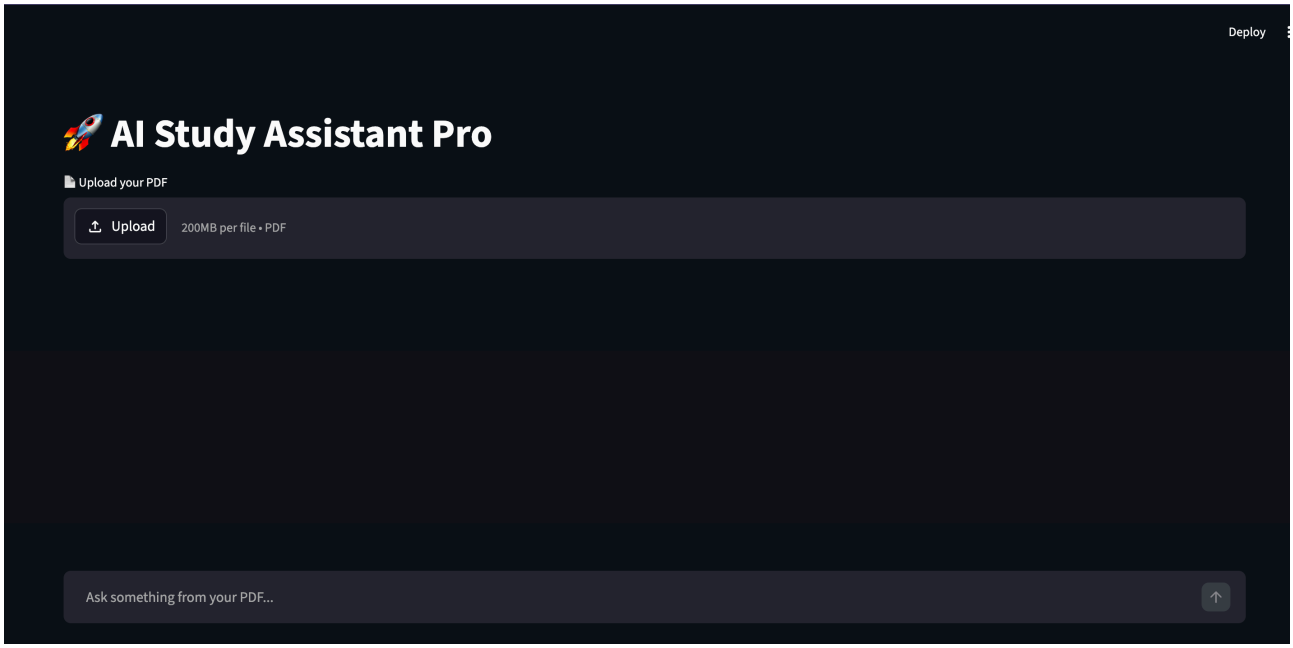
- **Python** – Core programming language
- **Streamlit** – For building the web-based user interface
- **LangChain** – For managing LLM workflows and chains
- **FAISS** – Vector database for storing embeddings
- **HuggingFace Transformers** – For running local language models
- **PyPDF Loader** – For extracting text from PDF files
- **Sentence Transformers** – For generating embeddings

These technologies were selected because they are efficient, widely used, and suitable for building scalable AI applications.

4. System Architecture

The system follows a modular architecture where each component performs a specific role. The workflow of the application is as follows:

1. The user uploads a PDF document through the interface
2. The system extracts text from the document
3. The text is split into smaller chunks
4. Each chunk is converted into vector embeddings
5. The embeddings are stored in a FAISS vector database
6. The user enters a query
7. The system retrieves the most relevant chunks from the database
8. The retrieved content is passed to the language model
9. The model generates a response based on the context
10. The response is displayed to the user



👉 *Upload a screenshot of your app interface showing PDF upload + chat UI*

5. Methodology

5.1 Data Processing

The first step involves processing the uploaded PDF file. The system uses a PDF loader to extract text from the document. Since large texts are difficult to process directly, the content is divided into smaller chunks using a text splitter.

This chunking process ensures:

- Better retrieval accuracy
- Faster processing
- Efficient embedding generation

5.2 Embedding Generation

After splitting the text, each chunk is converted into numerical vectors using a pre-trained embedding model (`all-MiniLM-L6-v2`).

These embeddings represent semantic meaning, allowing the system to:

- Understand context
- Perform similarity search
- Retrieve relevant information

5.3 Vector Database (FAISS)

The generated embeddings are stored in a FAISS vector database. FAISS is optimized for fast similarity search and is widely used in AI applications.

When a user asks a question:

- The query is converted into an embedding
- FAISS finds the closest matching chunks
- Relevant content is retrieved

5.4 Retrieval-Augmented Generation (RAG)

RAG is the core concept used in this project. Instead of relying only on the language model, the system first retrieves relevant information from the document and then generates a response.

This approach improves:

- Accuracy
- Relevance
- Context-awareness

5.5 Language Model Integration

The system uses a HuggingFace-based model (`flan-t5-base`) running locally. This ensures:

- No dependency on paid APIs
- Better reliability during demonstration
- Cost-free execution

The model receives:

- User query
- Retrieved document context

And generates a meaningful answer.

5.6 Conversational Memory

To enhance user interaction, the system includes a memory module. This allows the chatbot to remember previous messages and maintain context across multiple queries.

For example:

- Follow-up questions become more meaningful
- Conversations feel more natural

5.7 Guardrails Implementation

Basic guardrails are implemented to prevent misuse. The system checks user queries for unsafe or irrelevant content such as:

- Hacking-related instructions
- Malicious prompts
- Attempts to bypass system behavior

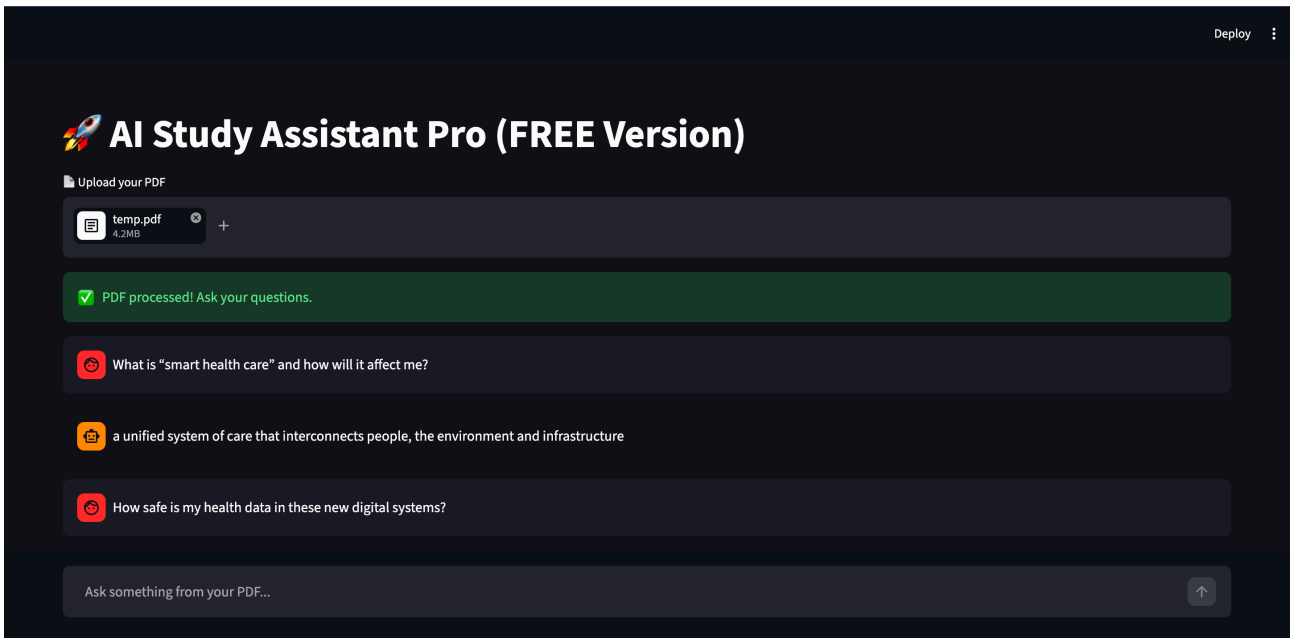
If such queries are detected, the system blocks them and shows a warning.

6. Implementation Details

The project is divided into two main files:

app.py

- Handles user interface using Streamlit



- Manages file uploads
- Displays chat messages

utils.py

- Processes PDF files
- Creates embeddings and vector database
- Builds the RAG pipeline
- Implements guardrails

The modular design improves readability and maintainability.

👉 *Add screenshot showing chat interaction (question + AI response)*

7. Results and Observations

The system was tested using multiple PDF documents related to academic topics. The following observations were made:

- The chatbot successfully answered questions based on uploaded content
- Responses were relevant and context-aware
- The system handled follow-up questions effectively due to memory
- Processing time was reasonable after initial model loading
- The interface was simple and easy to use

However, some limitations were observed:

- Initial model loading takes time
- Response quality depends on document clarity
- Local model is slightly less powerful than advanced APIs

8. Conclusion

This project successfully demonstrates the implementation of a **GenAI-based application using RAG architecture**. It combines multiple components such as embeddings, vector databases, and language models to create an intelligent system capable of answering questions from documents.

The use of a local model ensures reliability and cost-effectiveness, making the system suitable for real-world academic use. Additionally, features like memory and guardrails enhance usability and safety.

Overall, this project provided valuable hands-on experience in building AI systems and understanding how modern LLM-based applications work.

9. Future Enhancements

The system can be further improved by adding:

- Support for multiple file uploads
- Integration with web search APIs
- Voice input/output features
- Advanced guardrails using AI moderation
- Deployment on cloud platforms

These enhancements can make the system more powerful and scalable.